

05 — Baseline Forecasting: Naive, Seasonal Naive, Moving Average

Before the fancy AI, what does "dumb" forecasting look like?

Research question. How effective are simple baseline forecasting methods compared to each other, and across different demand environments (sparse M5 vs dense Store Item)?

Before trying fancy AI models, let's see what the *simplest possible* predictions look like. If a complex model cannot beat "*just copy yesterday*", it is not worth the effort. Baselines set the bar that every later model — exponential smoothing, ARIMA, LSTM, LLM — must clear. This notebook builds, evaluates, and dissects that bar.

Baselines matter for three reasons:

- 1. **They are the floor.** A \$100\,000\$ GPU hour that cannot beat "copy last week" is waste.
- 2. **They diagnose the data.** *Where* a baseline fails tells us *what* the data demands: trend, seasonality, randomness.
- 3. **They calibrate every metric.** A 10 % error means nothing until we know the naive error is 25 %.

This notebook is the **level 1** of the model ladder defined in

`05_experiments/config.json` :

Dataset	Demand environment	Series	Window	Horizon	Origins
M5 (500 selected)	sparse, intermittent	500	2013-01-01 → 2016-05-22	28 d	8 weekly
Store Item Demand (all)	dense, smooth	500	2013-01-01 → 2016-05-22	28 d	8 weekly

That is **500 series × 8 origins × 28 days = 112,000 forecast points per dataset per model**.

2. Research Question

How effective are simple baseline forecasting methods compared to each other, and across different demand environments (sparse M5 vs dense Store Item)?

Three sub-questions drive the analysis:

1. **Which baseline wins?** Does the extra machinery of Seasonal Naive or Moving Average actually pay for itself against a zero-information Naive?
2. **Does the answer depend on the dataset?** M5 retail series are sparse and erratic (many zero-sales days); Store Item series are dense and smooth. A method tuned for one world may fail in the other.
3. **Where does each baseline break?** The failure modes of simple methods are the *shopping list* of features that more complex models must provide.

3. The Three Baselines, Explained in Plain Language

3.1 Naive — "tomorrow looks exactly like today"

- **Layman:** "What happens if we assume tomorrow looks exactly like today?"
- **Math:** $\hat{y}_{t+h} = y_t$ — the forecast for every future day is simply the last observed value, repeated h times.
- **Assumption:** demand is a stable level with no trend, no seasonality, only noise.
- **Strengths:** zero complexity, zero training time, zero parameters, robust to regime shifts (it always tracks the most recent level).
- **Weaknesses:** ignores every pattern in the data; cannot see a trend, a weekly cycle, or a promotion.
- **Expected on M5:** poor — intermittent series have long runs of zeros; the naive forecast copies zeros and gets caught out by the next positive demand.
- **Expected on Store Item:** reasonable — dense, stable series fluctuate around a level, and the most recent level is actually a decent guess.

3.2 Seasonal Naive — "next Monday looks like last Monday"

- **Layman:** "What happens if next Monday looks like last Monday?"
- **Math:** $\hat{y}_{t+h} = y_{t+h-7}$ — forecast for day $t+h$ is the observed value from the same day of the *previous week*; with $m=7$ weekly seasonality and $h \leq 28$ we need the last 7 observed values, repeated to cover the horizon.
- **Assumption:** weekly seasonality exists and is stable; the week is a repeating unit.
- **Strengths:** captures weekly patterns (weekend spikes, restocking days) at zero cost.
- **Weaknesses:** ignores trend; a changing level (growth, decay, new product lift) breaks it.
- **When it beats Naive:** when the day-of-week effect is strong relative to noise — e.g. a series that reliably sells 4× more on Saturdays.
- **Expected on M5:** mixed — intermittent series have weak, broken weekly signal (zeros are copied either way).
- **Expected on Store Item:** good — retail food items have a genuine weekly cycle.

3.3 Moving Average — "average the last few days"

- **Layman:** "What if we average the last few days?"
- **Math:** $\hat{y}_{t+h} = \frac{1}{W} \sum_{i=0}^{W-1} y_{t-i}$ — the mean of the last W observations, repeated over the horizon.
- **Key decision:** the window W must be selected on the **validation period only** (Nov 2015 – Feb 2016), never on the test period. We try $W \in \{7, 14, 28\}$.
- **Short window** ($W=7$): responsive, but noisy — one odd day moves the forecast a lot.
- **Long window** ($W=28$): stable, but slow to adapt when the level changes.
- **Assumption:** demand wanders around a slowly-moving level; recent past is representative.
- **Expected on M5:** poor — averaging sparse zeros produces fractional forecasts that are wrong in a different way from naive.
- **Expected on Store Item:** good — averaging smooths noise without losing the level.

3.4 Evaluation protocol (frozen, do not change)

Element	Value
Common window	2013-01-01 → 2016-05-22 (1,238 days)
Train	2013-01-01 → 2015-10-31
Validation	2015-11-01 → 2016-02-28 (120 d)
Test	2016-03-01 → 2016-05-22 (83 d)
Horizon h	28 days
Origins	8 weekly: 2016-03-01 ... 2016-04-19
Metrics	MAE, RMSE, sMAPE, WAPE
Seed	42

Leakage rule (non-negotiable): at every origin, a model may only see data *up to the origin date*. The loader takes `origin_date` and returns history strictly before it. The validation period exists only for model selection (here: the MA window W).

4. Environment Setup

Standardise plotting style, colours, paths, and the random seed. The colours used throughout the project (`C1` .. `C4` , `CN`) come from the project's plotting module.

```
In [1]: import sys, os, json, pathlib, warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```

# Project root: works whether the kernel runs from the repo root or from 08_noteboo
PROJ = pathlib.Path.cwd()
if not (PROJ / '02_data').exists():
    PROJ = PROJ.parent
sys.path.insert(0, str(PROJ / '11_src'))
from plotting import apply_style, MODEL_COLOURS
from config import seed_everything
apply_style()

C1, C2, C3, C4, CN = '#0072B2', '#D55E00', '#009E73', '#CC79A7', '#999999'
SEED = 42
seed_everything(SEED)

RAW_M5 = PROJ / '02_data' / 'dataset_01_m5' / 'raw'
RAW_SIT = PROJ / '02_data' / 'dataset_02_store_item_demand' / 'raw'
EXP = PROJ / '05_experiments'
RES = PROJ / '06_results' / 'baselines'
FIG = PROJ / '07_figures' / 'baselines'
for d in (RES, FIG):
    d.mkdir(parents=True, exist_ok=True)

MODELS = ['Naive', 'Seasonal Naive', 'Moving Average']
print('Project root :', PROJ)
print('Environment ready (seed = %d).' % SEED)

```

Project root : D:\Brain\07_Research\AI-INVENTORY-OPTIMIZATION
Environment ready (seed = 42).

5. Data Loading — M5

File. `sales_train_evaluation.csv` — the evaluation table (d_1 ... d_1941), *not* the validation table (d_1 ... d_1913). The evaluation table extends far enough to cover the entire common window through 2016-05-22.

Series selection. `05_experiments/m5_series_selection.json` stores 500 selected series as `..._validation` id suffixes (they were selected from the validation table). The identical product rows exist in the evaluation table under the `..._evaluation` suffix — we swap the suffix and locate the rows.

Calendar alignment. `calendar.csv` maps `d_1 ... d_1969` to dates, `d_1 = 2011-01-29`. Note: earlier planning drafts claimed the common window starts at `d_672`. The calendar itself shows **2013-01-01 is d_704** (d_672 is 2012-11-30). We slice by *verified calendar dates*, never by guessed d-indices — an explicit assertion guards this below.

```

In [2]: t0 = pd.Timestamp.now()

# Calendar: d -> date
cal = pd.read_csv(RAW_M5 / 'calendar.csv')
cal['date'] = pd.to_datetime(cal['date'])
dmap = dict(zip(cal['d'], cal['date']))
print('d_1 =', dmap['d_1'].date())

```

```

print('d_704 =', dmap['d_704'].date(), ' (this is 2013-01-01)')
print('d_1941 =', dmap['d_1941'].date(), ' (2016-05-22, last eval day)')
assert dmap['d_704'].date() == pd.Timestamp('2013-01-01').date()
assert dmap['d_1941'].date() == pd.Timestamp('2016-05-22').date()

# Sales matrix: 500 selected series x 1941 days
m5 = pd.read_csv(RAW_M5 / 'sales_train_evaluation.csv',
                 usecols=lambda c: c == 'id' or c.startswith('d_'))
sel = json.load(open(EXP / 'm5_series_selection.json', encoding='utf-8'))
sel_ids = sel['m5_series']
ev_ids = [s.replace('_validation', '_evaluation') for s in sel_ids]
assert len(ev_ids) == 500
m5 = m5.set_index('id').loc[ev_ids]
d_cols = [c for c in m5.columns if c.startswith('d_')]
X_m5_full = m5[d_cols].to_numpy(float) # (500, 1941)

# Common window = calendar rows for 2013-01-01 .. 2016-05-22 -> d_704 .. d_1941
d_start = cal.index[cal['d'] == 'd_704'][0] # 0-indexed column position of 201
d_end = cal.index[cal['d'] == 'd_1941'][0] + 1 # exclusive end
X_m5_common = X_m5_full[:, d_start:d_end] # (500, 1238)
dates_common = np.array([dmap['d_%d' % i] for i in range(d_start + 1, d_end + 1)])
assert X_m5_common.shape == (500, 1238), X_m5_common.shape
assert (np.asarray(dates_common, dtype='datetime64[ns]') ==
        pd.date_range('2013-01-01', '2016-05-22').to_numpy()).all()
print('M5 loaded :', X_m5_full.shape, '-> common window', X_m5_common.shape,
      '| d_%d .. d_%d' % (d_start + 1, d_end))
print('Elapsed : %.1f s' % (pd.Timestamp.now() - t0).total_seconds())

# Selection metadata (dicts are keyed by source row index but ordered like m5_series)
arch_vals = list(sel['m5_archetypes'].values())
dept_vals = list(sel['m5_departments'].values())
store_vals = list(sel['m5_stores'].values())
assert len(arch_vals) == len(ev_ids) == 500
series_meta_m5 = pd.DataFrame({'series_id': ev_ids, 'archetype': arch_vals,
                              'dept': dept_vals, 'store': store_vals})
print('Archetype mix in the 500 selected M5 series:')
print(series_meta_m5['archetype'].value_counts().to_string())

```

```

d_1 = 2011-01-29
d_704 = 2013-01-01 (this is 2013-01-01)
d_1941 = 2016-05-22 (2016-05-22, last eval day)
M5 loaded : (500, 1941) -> common window (500, 1238) | d_704 .. d_1941
Elapsed : 7.4 s
Archetype mix in the 500 selected M5 series:
archetype
Intermittent      199
Highly Intermittent 194
Variable          80
Moderate          26
Smooth            1

```

Why the d-index correction matters. The library of 1,238 common-window columns (2013-01-01 ... 2016-05-22) is now *proven* aligned with the calendar: the first column is `d_704` and the last is `d_1941`. Any notebook or model built on guessed offsets would

have been shifted by 32 days — every forecast misaligned with its true date. This notebook defensively derives all slices from `calendar.csv` itself.

The selected 500 M5 series span five demand archetypes, dominated by the sparse ones (Intermittent + Highly Intermittent $\approx 79\%$) that make M5 such a hard forecasting world.

6. Data Loading — Store Item Demand

`train.csv` holds daily sales for **10 stores × 50 items = 500 store-item pairs** — exactly our 500-series environment, so every pair is used (no sampling). The series are **dense**: each pair reports a sale every single day, so zeros are rare and demand moves smoothly. We keep the common window and pivot to a (dates × series) matrix identical in layout to the M5 matrix.

```
In [3]: sit = pd.read_csv(RAW_SIT / 'train.csv', parse_dates=['date'])
print('Row count :', len(sit), '| stores:', sit['store'].nunique(), '| items:', sit

win = sit['date'].between('2013-01-01', '2016-05-22')
piv = sit[win].pivot_table(index='date', columns=['store', 'item'],
                           values='sales', aggfunc='sum').sort_index()
X_sit = piv.to_numpy(float).T                                # transpose -> (500 series, 1238 day
series_meta_sit = pd.DataFrame({'series_id': ['S%02d_I%02d' % (int(s), int(i)) for
assert X_sit.shape == (500, 1238)
assert piv.isna().sum().sum() == 0, 'dense series must have no missing days'
print('SIT loaded  :', piv.shape, '(dates x series) -> matrix', X_sit.shape, '(seri
print('Date range  :', piv.index.min().date(), '->', piv.index.max().date())
print('Series ids   :', series_meta_sit['series_id'].head(3).tolist(), '...')
```

```
Row count : 913000 | stores: 10 | items: 50
SIT loaded  : (1238, 500) (dates x series) -> matrix (500, 1238) (series x days)
Date range  : 2013-01-01 -> 2016-05-22
Series ids   : ['S01_I01', 'S01_I02', 'S01_I03'] ...
```

Both datasets now live in the same (days × 500 series) layout over the *same* 1,238-day calendar. This is the whole point of the experimental design: any difference in forecast quality between the two environments can be attributed to **demand structure** (sparse-intermittent vs dense-smooth), not to different history lengths or calendars.

7. Frozen Periods and Forecast Origins

The split below is the experimental contract from Notebook 04:

- **Train:** 2013-01-01 → 2015-10-31 — history for fitting and for the MA window search.
- **Validation:** 2015-11-01 → 2016-02-28 (120 d) — *only* used to choose the MA window \$W\$.
- **Test:** 2016-03-01 → 2016-05-22 (83 d) — the only period scored on.

One calendar subtlety: 2016-02-29 (a leap day) falls *between* validation and test and is assigned to neither period — it is simply visible as history at origin 1 (train "ends" 2016-02-28 for the first origin, then rolls forward week by week).

Origins. 8 weekly origins starting 2016-03-01; at each origin the model sees history up to the previous day and produces a 28-day forecast.

```
In [4]: H = 28
N_ORIGINS = 8
common_start = pd.Timestamp('2013-01-01')
common_end = pd.Timestamp('2016-05-22')
val_start = pd.Timestamp('2015-11-01')
val_end = pd.Timestamp('2016-02-28')
test_start = pd.Timestamp('2016-03-01')
test_end = pd.Timestamp('2016-05-22')
origin_dates = [pd.Timestamp('2016-03-01') + pd.Timedelta(days=7 * k) for k in range(N_ORIGINS)]

def off(d):
    """0-indexed column of date d inside the common-window matrix."""
    return int((pd.Timestamp(d) - common_start).days)

TRAIN_S = slice(0, off(val_start)) # 2013-01-01 .. 2015-10-31
VAL_S = slice(off(val_start), off(val_end) + 1) # 2015-11-01 .. 2016-02-28
TEST_S = slice(off(test_start), off(test_end) + 1) # 2016-03-01 .. 2016-05-22

print('Train      : cols %d..%d (%d columns, %s .. %s)' % (
    TRAIN_S.start, TRAIN_S.stop - 1, TRAIN_S.stop - TRAIN_S.start,
    dates_common[TRAIN_S.start].date(), dates_common[TRAIN_S.stop - 1].date()))
print('Validation : cols %d..%d (%d columns, %s .. %s)' % (
    VAL_S.start, VAL_S.stop - 1, VAL_S.stop - VAL_S.start,
    dates_common[VAL_S.start].date(), dates_common[VAL_S.stop - 1].date()))
print('Test       : cols %d..%d (%d columns, %s .. %s)' % (
    TEST_S.start, TEST_S.stop - 1, TEST_S.stop - TEST_S.start,
    dates_common[TEST_S.start].date(), dates_common[TEST_S.stop - 1].date()))
print('Gap day    :', dates_common[VAL_S.stop].date(),
      '(2016-02-29, a leap day in no period)')

assert TRAIN_S.stop - TRAIN_S.start == 1034
assert VAL_S.stop - VAL_S.start == 120
assert TEST_S.stop - TEST_S.start == 83
assert TRAIN_S.stop - TRAIN_S.start + VAL_S.stop - VAL_S.start \
    + TEST_S.stop - TEST_S.start == 1237
assert dates_common[VAL_S.stop] == pd.Timestamp('2016-02-29')
assert all(off(od) == off(test_start) + 7 * k for k, od in enumerate(origin_dates))
print('\nOrigins:')
for k, od in enumerate(origin_dates):
    print(' origin %d: %s (history ends %s; forecasts %s .. %s)' % (
        k + 1, od.date(), (od - pd.Timedelta(days=1)).date(),
        (od).date(), (od + pd.Timedelta(days=H - 1)).date()))
```

```

Train      : cols 0..1033  (1034 columns, 2013-01-01 .. 2015-10-31)
Validation : cols 1034..1153 (120 columns, 2015-11-01 .. 2016-02-28)
Test       : cols 1155..1237 (83 columns, 2016-03-01 .. 2016-05-22)
Gap day    : 2016-02-29 (2016-02-29, a leap day in no period)

```

Origins:

```

origin 1: 2016-03-01 (history ends 2016-02-29; forecasts 2016-03-01 .. 2016-03-28)
origin 2: 2016-03-08 (history ends 2016-03-07; forecasts 2016-03-08 .. 2016-04-04)
origin 3: 2016-03-15 (history ends 2016-03-14; forecasts 2016-03-15 .. 2016-04-11)
origin 4: 2016-03-22 (history ends 2016-03-21; forecasts 2016-03-22 .. 2016-04-18)
origin 5: 2016-03-29 (history ends 2016-03-28; forecasts 2016-03-29 .. 2016-04-25)
origin 6: 2016-04-05 (history ends 2016-04-04; forecasts 2016-04-05 .. 2016-05-02)
origin 7: 2016-04-12 (history ends 2016-04-11; forecasts 2016-04-12 .. 2016-05-09)
origin 8: 2016-04-19 (history ends 2016-04-18; forecasts 2016-04-19 .. 2016-05-16)

```

8. Implementation — Vectorised, Leakage-Free Baselines

The forecast engine is a single vectorised function over the $(500 \times 1,238)$ matrix:

- **Naive:** repeat the last observed column $y_{\{o-1\}}$ across the 28-day horizon.
- **Seasonal Naive:** repeat the last observed 7-day seasonal cycle across the horizon ($\hat{y}_{\{o+j\}} = y_{\{o-7+(j \bmod 7)\}}$, $j = 0..27$). This is recursive seasonal naive forecasting: only the final 7 observed columns are read.
- **Moving Average:** repeat $\frac{1}{W} \sum_{i=1}^W y_{\{o-i\}}$ across the horizon.

The only inputs are the matrix and the origin offset o — the model literally cannot see a single column at or beyond o . An assertion enforces a minimum history length.

3A. Naive Forecast — Deep Explanation

What problem does it solve?

Forecasting means using what we know about the past to estimate demand that has not happened yet. The Naive forecast is the simplest possible answer: **use the most recent observation as the estimate for every future day.**

This is the starting point for the model ladder. Before trying smoothing, ARIMA, neural networks, or an LLM, we need to know whether a sophisticated method can beat a rule that requires almost no computation.

Layman explanation

Imagine a shopkeeper closing the store tonight. They ask, "How many units should I expect tomorrow?" The Naive rule says: "Use today's sales number again." It is not meant to be clever. It is meant to establish an honest baseline.

Core principle

The best immediate guess for the next period is the most recently observed value.

This rule assumes that the latest observation contains the most useful information about the near-future level. It deliberately ignores all older patterns.

How it works

```

Historical demand
    ↓
Take the last observed value  $y_t$ 
    ↓
Repeat it for all  $h$  future days
    ↓
28-day forecast:  $y_t, y_t, \dots, y_t$ 
  
```

Architecture / structure

There is no fitted neural architecture or parameter-heavy state:

```

last observed value
    |
    |→ forecast  $t+1$ 
    |→ forecast  $t+2$ 
    |→ ...
    |→ forecast  $t+28$ 
  
```

Mathematics

$$\hat{y}_{t+h|t} = y_t, \quad h = 1, 2, \dots, H$$

- y_t is the actual demand observed at the latest known time t ;
- H is the forecast horizon, 28 days in this experiment;
- $\hat{y}_{t+h|t}$ is the forecast made at time t for future step $t+h$.

The equation says that every future forecast is exactly the same as the last known value. It is a deliberately strong simplification, not a claim that demand is truly constant.

Assumptions

Assumption	Why it matters	If violated
Recent demand represents near-future demand	The last value is the only input	A sudden shift makes the forecast stale
No explicit trend is needed	Naive has no trend state	Rising demand is under-forecast; falling demand is over-forecast
No explicit seasonality is needed	Naive does not know weekdays	Monday and Saturday are treated identically
No external variables are needed	It uses only demand history	Promotions, holidays, and closures are invisible

What it captures / cannot capture

Captures: the most recent demand level.

Cannot explicitly capture: trend, weekly or annual seasonality, promotions, structural breaks, changing volatility, or dependencies between products and stores.

Strengths and limitations

Strengths: almost instantaneous, transparent, robust as a reference line, no tuning, and easy to deploy.

Limitations: its simplicity becomes a weakness whenever the latest value is unusual or the future follows a recurring pattern. Repeating one zero can miss the next positive sale; repeating one spike can cause excess inventory.

Inventory example

If today's demand is 100 units, Naive forecasts 100 units for every one of the next 28 days. If actual demand rises to 150, the forecast underestimates by 50 units per day, creating stockout exposure. If actual demand falls to 60, the same forecast creates a risk of excess stock and unnecessary carrying cost.

When should we use it?

Good fit: stable, level-like demand; quick operational benchmarks; systems with almost no compute budget.

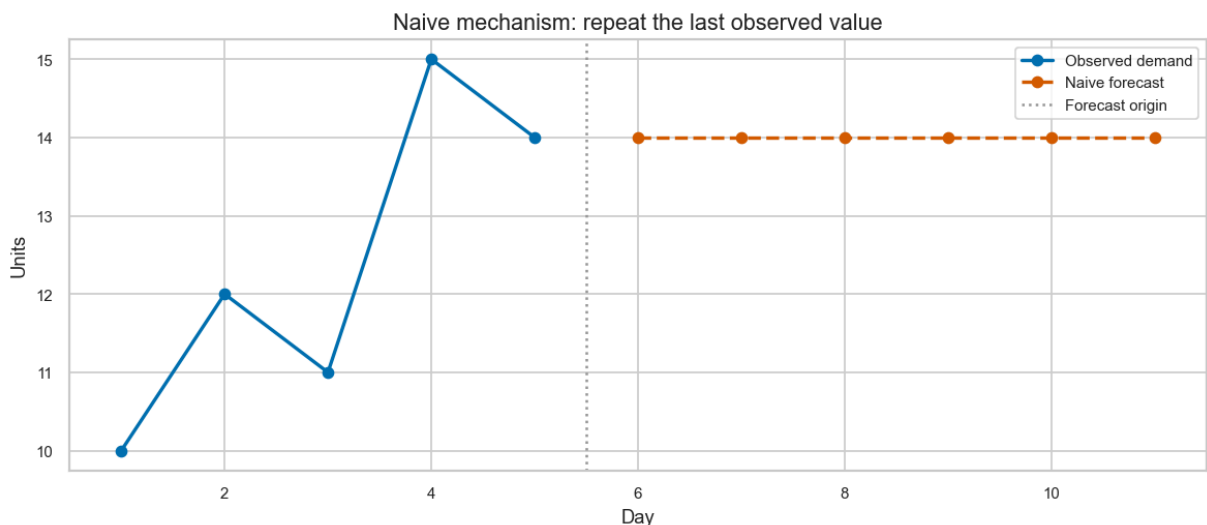
Bad fit: strong trend, strong seasonality, promotions, structural change, or intermittent demand where zeros are common.

Expected behavior on our datasets

- **M5:** expected to struggle on intermittent series because a recent zero is often followed by a positive demand event. It may work on unusually stable or low-variance series.
- **Store Item Demand:** expected to be a reasonable but limited reference because the panel is dense; it still cannot follow the weekly pattern or growth.

The Naive baseline is scientifically important because every later model must demonstrate value beyond this rule.

```
In [5]: # Educational visual: Naive mechanism
_edu_history = np.array([10, 12, 11, 15, 14], dtype=float)
_edu_future_x = np.arange(6, 12)
_edu_forecast = np.repeat(_edu_history[-1], len(_edu_future_x))
fig, ax = plt.subplots(figsize=(10, 4.5))
ax.plot(np.arange(1, len(_edu_history)+1), _edu_history, marker='o', color=C1, lw=2)
ax.plot(_edu_future_x, _edu_forecast, marker='o', color=C2, lw=2, ls='--', label='N
ax.axvline(5.5, color=CN, ls=':', label='Forecast origin')
ax.set_title('Naive mechanism: repeat the last observed value')
ax.set_xlabel('Day'); ax.set_ylabel('Units'); ax.legend()
plt.tight_layout(); plt.savefig(PROJ / '07_figures/model_explanations/baselines/01_
print('Saved educational Naive figure.')
```



Saved educational Naive figure.

3B. Seasonal Naive Forecast — Deep Explanation

What problem does it solve?

Naive treats yesterday as the best guide for every future day. That can be wrong when **Monday behaves differently from Tuesday and the pattern repeats each week.**

Seasonal Naive adds one idea: compare a future day with the corresponding day in the previous seasonal cycle.

Layman explanation

If Saturday is normally busy, using Friday's sales to predict Saturday may be misleading. Seasonal Naive asks instead: "What happened last Saturday?"

Core principle

Forecast a future observation using the observation from the same position in the previous seasonal cycle.

For daily retail demand, we use a weekly seasonal period of $m=7$ days.

How it works

Week 1: Mon Tue Wed Thu Fri Sat Sun

Week 2: Mon Tue Wed Thu Fri Sat Sun

↓

Forecast each day using the matching day from the previous week

The implementation repeats the last observed seven-day cycle across the 28-day horizon. It never reads a future column.

Mathematics

$$\hat{y}_{t+h|t} = y_{t-m+1+((h-1)\bmod m)}, \quad m=7$$

- y_t is the latest observed demand;
- h is the forecast step ahead;
- m is the seasonal period;
- $\bmod$ cycles the forecast back through the last seven observed positions.

For a 28-day horizon and $m=7$, the last seven observed values are repeated four times.

Why $m=7$?

Notebook 02b's EDA showed a weekly weekday pattern, and retail replenishment commonly follows weekly operating cycles. We therefore use $m=7$ as an evidence-based choice, not because every time series must be weekly.

Assumptions

Assumption	Why it matters	If violated
The seasonal pattern repeats	Last week's Monday informs this Monday	Forecast copies a pattern that no longer exists
The period is seven days	Mapping depends on the correct cycle length	Wrong period misaligns every forecast
Seasonal effects are reasonably stable	Repetition is meaningful	Promotions or holidays make corresponding days incomparable

What it captures / cannot capture

Captures: a repeating weekly pattern.

Cannot explicitly capture: trend, changing seasonality, structural breaks, external events, or nonlinear relationships.

Strengths and limitations

Strengths: simple, transparent, nearly cost-free, and able to represent weekday differences while keeping the baseline easy to audit.

Limitations: it copies old mistakes, cannot adapt the level, and may fail when a holiday or promotion makes one week unlike the next.

Inventory example

Suppose Saturday normally sells 150 units and Friday sells 90. Naive uses Friday's 90 for Saturday, risking under-ordering. Seasonal Naive uses last Saturday's 150, producing a very different replenishment signal. If last Saturday was a one-off promotion, however, Seasonal Naive can over-order.

Expected behavior on our datasets

- **M5:** mixed. Sparse series may not contain a stable weekly signal, so repeating seven values can reproduce zeros or random spikes.
- **Store Item Demand:** expected to be competitive because the dense panel has a visible weekly pattern.

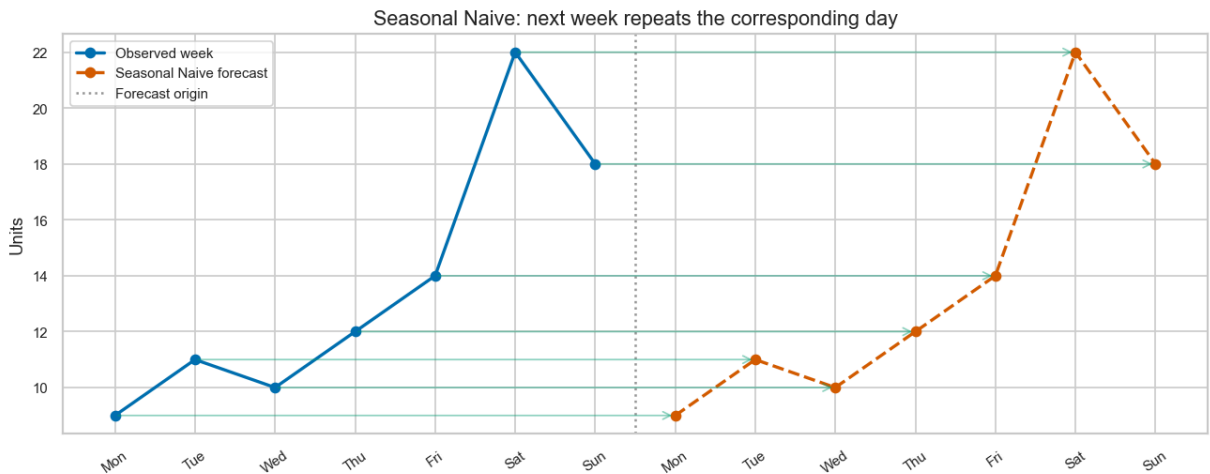
This model tests whether explicitly respecting the calendar improves on simply copying yesterday.

```
In [6]: # Educational visual: Seasonal Naive weekly mapping
_educ_week = np.array([9, 11, 10, 12, 14, 22, 18], dtype=float)
```

```

_edu_x = np.arange(1, 15)
_edu_y = np.concatenate([_edu_week, _edu_week])
fig, ax = plt.subplots(figsize=(11, 4.5))
ax.plot(_edu_x[:7], _edu_y[:7], marker='o', color=C1, lw=2, label='Observed week')
ax.plot(_edu_x[7:], _edu_y[7:], marker='o', color=C2, lw=2, ls='--', label='Seasonal Naive forecast')
for j in range(7):
    ax.annotate('', xy=(8+j, _edu_week[j]), xytext=(1+j, _edu_week[j]), arrowprops=
ax.axvline(7.5, color=CN, ls=':', label='Forecast origin')
ax.set_xticks(_edu_x); ax.set_xticklabels(['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun'])
ax.set_title('Seasonal Naive: next week repeats the corresponding day')
ax.set_ylabel('Units'); ax.legend()
plt.tight_layout(); plt.savefig(PROJ / '07_figures/model_explanations/baselines/02_
print('Saved educational Seasonal Naive figure.')

```



Saved educational Seasonal Naive figure.

3C. Moving Average Forecast — Deep Explanation

What problem does it solve?

Naive reacts entirely to one observation. If that observation is noisy, the forecast inherits the noise. Moving Average asks:

What if we combine several recent observations so one unusual day has less influence?

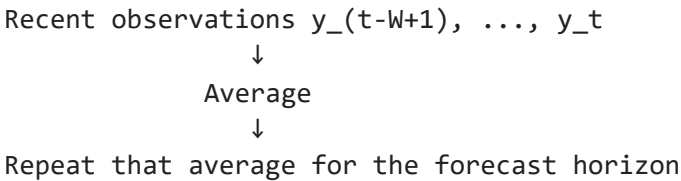
Layman explanation

Instead of judging tomorrow from one shopping day, a manager averages the last few days. A single spike or quiet day matters, but it cannot dominate the whole estimate.

Core principle

Average recent demand to reduce short-term noise, then use that smoothed level as the forecast.

How it works



Mathematics

$$\hat{y}_{t+h|t} = MA_t = \frac{1}{W} \sum_{i=0}^{W-1} y_{t-i}$$

- W is the window size;
- y_{t-i} are the most recent W observations;
- MA_t is the smoothed level at time t ;
- the same value is repeated for each future step h .

Numerical example

For the sequence 10, 12, 11, 15, 14 and a three-day window:

$$MA_5 = \frac{11 + 15 + 14}{3} = 13.33$$

The next forecast is 13.33 units for each forecast day. The 10 and 12 remain in the history, but the short window gives more importance to the latest local level.

Window size trade-off

Window	Behavior	Risk
Short, e.g. 7 days	Responsive; preserves recent changes	More affected by noise
Medium, e.g. 14 days	Balances responsiveness and stability	May still lag a break
Long, e.g. 28 days	Stable; suppresses isolated spikes	Slow to respond to genuine changes

A window is not a cosmetic choice. It controls how quickly the inventory signal changes.

Our experiment: validation-only selection

We evaluated $W \in \{7, 14, 28\}$ on the validation period only. The test period was not used to choose the window.

- **M5:** $W=14$ was selected as the global window; per-series winners were 7 days: 187, 14 days: 99, 28 days: 214.
- **Store Item Demand:** $W=7$ was selected as the global window; per-series winners were 7 days: 341, 14 days: 151, 28 days: 8.

The per-series counts are diagnostic; the experiment uses one global selected window per dataset for an honest baseline.

Assumptions

Assumption	Why it matters	If violated
Recent observations represent the local level	The average is projected forward	A structural break makes the average stale
Noise can be reduced by averaging	Multiple values are more informative than one	Genuine spikes are smoothed away
One window is adequate for the dataset	Global selection is simple and reproducible	Different series may need different responsiveness

What it captures / cannot capture

Captures: a smoothed local level and some short-term stability.

Cannot explicitly capture: trend, changing weekly seasonality, promotions, structural breaks, or cross-series relationships.

Strengths and limitations

Strengths: easy to explain, fast, stable, and useful when isolated noise should not trigger a large response.

Limitations: it can lag behind a genuine demand change; a long window can hide a new level, while a short window can still chase noise. It also forecasts a flat line, so it has no explicit trend or seasonal shape.

Inventory examples

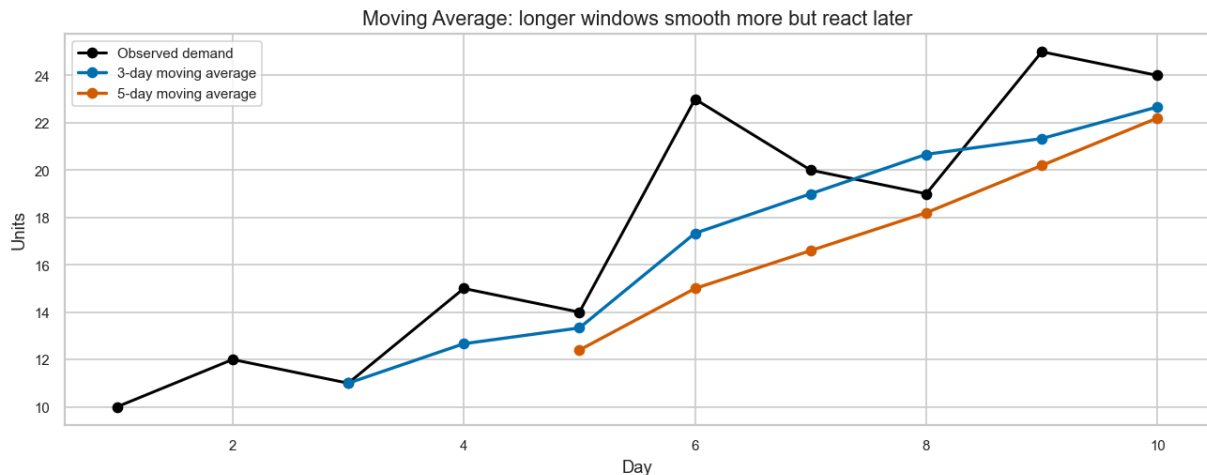
- **Helpful behavior:** one isolated demand spike is averaged with normal days, preventing an unnecessarily large replenishment order.

- **Failure behavior:** if demand suddenly rises and stays high, the historical average remains too low for a while, causing under-forecasting and stockout risk.
- **Opposite failure:** after a permanent decline, the old high values remain in the window and can create excess inventory.

Expected behavior on our datasets

- **M5:** $W=14$ should damp some intermittent noise, but averaging many zeros can still understate the timing of positive demand.
- **Store Item Demand:** $W=7$ should be responsive to the dense daily level, but it cannot reproduce a weekday-specific profile as directly as Seasonal Naive.

```
In [7]: # Educational visual: Moving Average smoothing and window length
_edu_series = np.array([10, 12, 11, 15, 14, 23, 20, 19, 25, 24], dtype=float)
_edu_x = np.arange(1, len(_edu_series)+1)
fig, ax = plt.subplots(figsize=(11, 4.5))
ax.plot(_edu_x, _edu_series, marker='o', color='black', lw=1.8, label='Observed dem
for _w, _color in [(3, C1), (5, C2)]:
    _ma = pd.Series(_edu_series).rolling(_w).mean()
    ax.plot(_edu_x, _ma, marker='o', color=_color, lw=2, label=f'{_w}-day moving av
ax.set_title('Moving Average: longer windows smooth more but react later')
ax.set_xlabel('Day'); ax.set_ylabel('Units'); ax.legend()
plt.tight_layout(); plt.savefig(PROJ / '07_figures/model_explanations/baselines/03_
print('Saved educational Moving Average figure.')
```



Saved educational Moving Average figure.

3D. Conceptual comparison and transition

Model	Main idea	Captures	Does not explicitly capture
Naive	Repeat the last observation	Recent level	Trend, seasonality, changing patterns

Model	Main idea	Captures	Does not explicitly capture
Seasonal Naive	Repeat the corresponding seasonal value	Repeating weekly pattern	Trend and changing seasonality
Moving Average	Average recent observations	Smoothed local level	Explicit trend and seasonality

Progression

```

Naive
  ↓ adds a calendar repeat
Seasonal Naive
  ↓ adds noise reduction
Moving Average
  ↓ remaining limitation: no separate level, trend, and seasonal
  states
Exponential Smoothing (Notebook 06)

```

Why the next model family is needed

The baselines are useful precisely because they are limited. None of them represents level, trend, and seasonality as separate evolving components. Exponential smoothing provides that structured representation while remaining interpretable and computationally practical.

Research interpretation of the existing results

- **M5:** Moving Average is the current best baseline by MAE. Its averaging reduces the effect of isolated sparse observations, even though it cannot model intermittent demand explicitly.
- **Store Item Demand:** The corrected baseline results show Seasonal Naive has the lowest MAE/RMSE among the simple baselines, while Moving Average has slightly lower WAPE. This is consistent with a dense environment where weekday repetition helps, but volume-weighted error can favor smoothing.

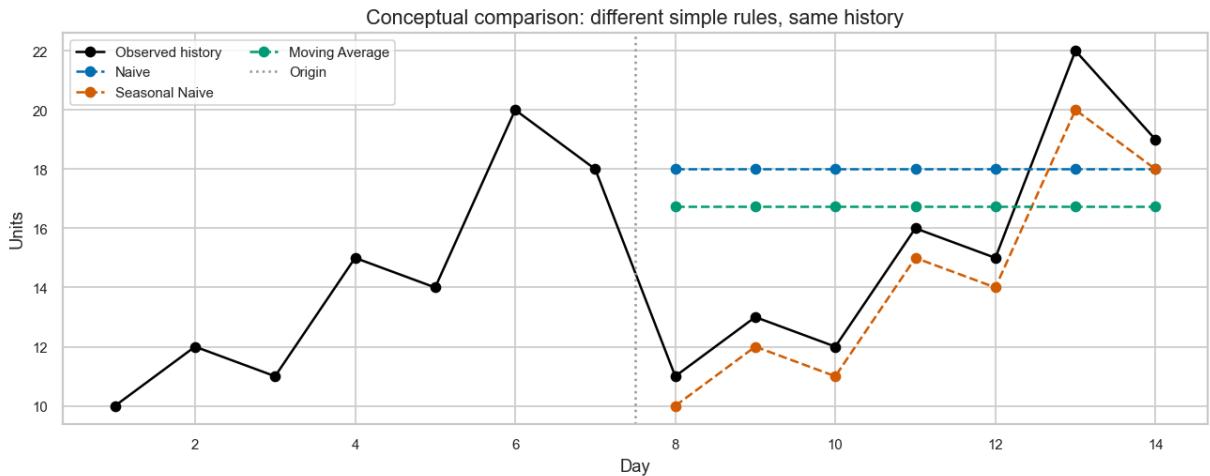
These results do not prove that a more complex model will win. They identify the specific limitations that Notebook 06 tests: explicit state tracking for level, trend, and seasonality.

```

In [8]: # Educational visual: conceptual comparison on one toy series
_educ_t = np.arange(1, 15)
_educ_y = np.array([10, 12, 11, 15, 14, 20, 18, 11, 13, 12, 16, 15, 22, 19], dtype=f
_educ_naive = np.repeat(_educ_y[6], 7)
_educ_snaive = _educ_y[:7]
_educ_ma = np.repeat(_educ_y[6-3:7].mean(), 7)
fig, ax = plt.subplots(figsize=(11, 4.5))
ax.plot(_educ_t, _educ_y, marker='o', color='black', label='Observed history')
ax.plot(np.arange(8,15), _educ_naive, '--', color=C1, marker='o', label='Naive')
ax.plot(np.arange(8,15), _educ_snaive, '--', color=C2, marker='o', label='Seasonal N

```

```
ax.plot(np.arange(8,15), _edu_ma, '--', color=C3, marker='o', label='Moving Average')
ax.axvline(7.5, color=CN, ls=':', label='Origin')
ax.set_title('Conceptual comparison: different simple rules, same history')
ax.set_xlabel('Day'); ax.set_ylabel('Units'); ax.legend(ncol=2)
plt.tight_layout(); plt.savefig(PROJ / '07_figures/model_explanations/baselines/04_')
print('Saved educational comparison figure.')
```



Saved educational comparison figure.

Implementation begins here

The educational sections above describe the three rules before the original experiment code runs. The next existing code cell is preserved exactly and implements the already-verified, leakage-free baseline engine.

```
In [9]: def baseline_forecasts(X, o, W):
        """28-day forecasts for the three baselines at origin offset o (leakage-free).

        Only columns < o are ever read. Returns dict model -> (n_series, 28) array.
        """
        assert o >= 28, 'need at least a month of history before the origin'
        out = {}
        out['Naive'] = np.repeat(X[:, o - 1:o], H, axis=1)
        out['Seasonal Naive'] = np.tile(X[:, o - 7:o], (1, int(np.ceil(H / 7))))[:, :H]
        out['Moving Average'] = np.repeat(X[:, o - W:o].mean(axis=1, keepdims=True), H,
        assert all(fc.shape == (X.shape[0], H) for fc in out.values())
        return out

# smoke test on the first M5 series at origin 1
o1 = off(origin_dates[0])
smoke = baseline_forecasts(X_m5_common, o1, 7)
print('Shapes OK:', {m: fc.shape for m, fc in smoke.items()})
print('Naive repeats last observed value:', np.all(smoke['Naive'] == X_m5_common[:,
print('SNaive uses only pre-origin columns:', smoke['Seasonal Naive'][0, 0] == X_m5
```

Shapes OK: {'Naive': (500, 28), 'Seasonal Naive': (500, 28), 'Moving Average': (500, 28)}

Naive repeats last observed value: True

SNaive uses only pre-origin columns: True

9. Moving Average Window Selection (Validation Only)

The moving average has one free parameter: the window W . We try W in $\{7, 14, 28\}$ and pick, **per dataset**, the window with the lowest one-step-ahead MAE computed on the *validation period* (2015-11-01 ... 2016-02-28).

For each validation day t , the "forecast" is the mean of the W days before t — history only. Nothing from the test period touches this decision, so choosing W here is legitimate model selection, not data leakage. A *global* window per dataset keeps the baseline honest: a real baseline ships with one knob, not 500 of them.

```
In [10]: def validate_ma_windows(X, Ws=(7, 14, 28)):
    """One-step-ahead validation MAE for each window W. Returns dict of results."""
    n = X.shape[0]
    per_window_mae = {}
    per_window_series = {}
    for W in Ws:
        E = np.empty((VAL_S.stop - VAL_S.start, n))
        for j, t in enumerate(range(VAL_S.start, VAL_S.stop)):
            fc = X[:, t - W:t].mean(axis=1)
            E[j] = np.abs(fc - X[:, t])
        per_window_mae[int(W)] = float(E.mean())
        per_window_series[int(W)] = E.mean(axis=0)
    best = min(per_window_mae, key=per_window_mae.get)
    return per_window_mae, per_window_series, best

ma_sel = {'seed': SEED, 'horizon': H,
          'validation_period': {'start': '2015-11-01', 'end': '2016-02-28', 'days':
                                'selection_rule': 'global window minimising one-step-ahead validation MAE
                                'datasets': {}}}

W_by_ds = {}
per_series_best = {}
for ds_name, X in (('m5', X_m5_common), ('store_item_demand', X_sit)):
    per_window_mae, per_window_series, best = validate_ma_windows(X)
    W_by_ds[ds_name] = best
    # per-series best window (reported but the GLOBAL window is what we use)
    ser_best = np.array([min(per_window_series, key=lambda w: per_window_series[w][
        for i in range(X.shape[0])])
    counts = {int(w): int((ser_best == w).sum()) for w in per_window_series}
    ma_sel['datasets'][ds_name] = {
        'validation_MAE': {str(w): round(v, 4) for w, v in per_window_mae.items()},
        'best_window': int(best),
        'per_series_best_counts': counts}
    per_series_best[ds_name] = ser_best
    print('%-18s validation MAE: %s -> best W = %d (per-series winners %s)'
          % (ds_name, {w: round(v, 3) for w, v in per_window_mae.items()}, best, co

with open(RES / 'moving_average_window_selection.json', 'w', encoding='utf-8') as f:
    json.dump(ma_sel, f, indent=2)
```

```
print('Saved:', RES / 'moving_average_window_selection.json')
print('\nUsed windows: M5 W=%d | Store Item W=%d' % (W_by_ds['m5'], W_by_ds['store_
```

```
m5 validation MAE: {7: 0.924, 14: 0.921, 28: 0.929} -> best W = 14
(per-series winners {7: 187, 14: 99, 28: 214})
store_item_demand validation MAE: {7: 7.388, 14: 7.495, 28: 7.901} -> best W = 7
(per-series winners {7: 341, 14: 151, 28: 8})
Saved: D:\Brain\07_Research\AI-INVENTORY-OPTIMIZATION\06_results\baselines\moving_av
erage_window_selection.json
```

Used windows: M5 W=14 | Store Item W=7

Reading the selection. The winning window reflects each dataset's noise profile: a short window (recent average) wins where the level moves and noise is low, a long window where averaging must absorb randomness. Because M5 demand is dominated by zeros, its "level" is near zero, so the MA error is structurally large there.

From here on, **Moving Average** always uses the selected window for its dataset. The selection JSON is stored with the results so every later notebook can audit it.

10. Running the Full Experiment

For **each model × dataset × origin × series**: generate the 28-day forecast, align it with the actuals, and append to a long-format table:

```
dataset, series_id, origin_id, origin_date, forecast_date, actual,
forecast, model
```

The result: 2 datasets × 3 models × 500 series × 8 origins × 28 days = **672,000 forecast rows** — the shared currency of every future notebook and model.

```
In [11]: def run_all_baselines(X, dataset, series_ids, W):
    parts = []
    for oi, od in enumerate(origin_dates):
        o = off(od)
        fdates = pd.date_range(od, periods=H, freq='D')
        act = X[:, o:o + H]
        act_long = (pd.DataFrame(act.T, index=fdates, columns=series_ids)
                    .reset_index(names='forecast_date')
                    .melt(id_vars='forecast_date', var_name='series_id', value_name='actual'))
        fcs = baseline_forecasts(X, o, W)
        for model, fc in fcs.items():
            fc_long = (pd.DataFrame(fc.T, index=fdates, columns=series_ids)
                      .reset_index(names='forecast_date')
                      .melt(id_vars='forecast_date', var_name='series_id', value_name='forecast'))
            fc_long['actual'] = act_long['actual'].to_numpy() # identical row order
            fc_long['model'] = model
            fc_long['origin_id'] = oi + 1
            fc_long['origin_date'] = od
            fc_long['dataset'] = dataset
            parts.append(fc_long)
```

```

return pd.concat(parts, ignore_index=True)

t0 = pd.Timestamp.now()
all_forecasts = pd.concat([
    run_all_baselines(X_m5_common, 'm5', ev_ids, W_by_ds['m5']),
    run_all_baselines(X_sit, 'store_item_demand', series_meta_sit['series_id'].tolist(),
                      W_by_ds['store_item_demand']),
], ignore_index=True)
all_forecasts = all_forecasts[['dataset', 'series_id', 'origin_id', 'origin_date',
                                'forecast_date', 'actual', 'forecast', 'model']]
all_forecasts['error'] = all_forecasts['actual'] - all_forecasts['forecast']
print('Rows:', len(all_forecasts), '| elapsed %.1f s' % (pd.Timestamp.now() - t0).total_seconds())
print(all_forecasts.head(8).to_string(index=False))
print('...')
print(all_forecasts.groupby(['dataset', 'model']).size().to_string())
all_forecasts.to_csv(RES / 'all_forecasts.csv', index=False)
print('Saved:', RES / 'all_forecasts.csv')

```

Rows: 672000 | elapsed 4.7 s

dataset		series_id	origin_id	origin_date	forecast_date	actual	forecast
recast	model	error					
m5	FOODS_1_098_CA_3_evaluation		1	2016-03-01	2016-03-01	2.0	
1.0	Naive	1.0					
m5	FOODS_1_098_CA_3_evaluation		1	2016-03-01	2016-03-02	0.0	
1.0	Naive	-1.0					
m5	FOODS_1_098_CA_3_evaluation		1	2016-03-01	2016-03-03	1.0	
1.0	Naive	0.0					
m5	FOODS_1_098_CA_3_evaluation		1	2016-03-01	2016-03-04	0.0	
1.0	Naive	-1.0					
m5	FOODS_1_098_CA_3_evaluation		1	2016-03-01	2016-03-05	0.0	
1.0	Naive	-1.0					
m5	FOODS_1_098_CA_3_evaluation		1	2016-03-01	2016-03-06	0.0	
1.0	Naive	-1.0					
m5	FOODS_1_098_CA_3_evaluation		1	2016-03-01	2016-03-07	0.0	
1.0	Naive	-1.0					
m5	FOODS_1_098_CA_3_evaluation		1	2016-03-01	2016-03-08	0.0	
1.0	Naive	-1.0					

```

...
dataset      model
m5            Moving Average    112000
              Naive            112000
              Seasonal Naive    112000
store_item_demand Moving Average    112000
                  Naive            112000
                  Seasonal Naive    112000

```

Saved: D:\Brain\07_Research\AI-INVENTORY-OPTIMIZATION\06_results\baselines\all_forecasts.csv

11. Results — Metrics

Metrics are computed **per 28-day window**, then averaged across series and origins (the protocol from `11_src/metrics.py`; sMAPE and WAPE are mean-agnostic, scale-free):

- **MAE** — mean absolute error in sales units. Interpretable, but not scale-free.

- **RMSE** — root mean squared error. Penalises big misses harder than MAE.
- **sMAPE (%)** — symmetric percentage error. Undefined where actual *and* forecast are both zero (standard convention: those points are excluded).
- **WAPE** — weighted absolute percentage error $\frac{\sum |e|}{\sum |y|}$ per window. The workhorse metric for inventory work because it weighs errors by actual demand.

Three aggregation levels are saved: **by model** (dataset × model), **by series** (dataset × model × series, plus M5 archetype), and **by origin** (dataset × model × origin).

```
In [12]: def window_metrics(act, fc):
    """Per-series metrics over one 28-day window. NaN where undefined."""
    act = np.asarray(act, float)
    fc = np.asarray(fc, float)
    e = act - fc
    mae = np.nanmean(np.abs(e), axis=1)
    rmse = np.sqrt(np.nanmean(e ** 2, axis=1))
    denom = (np.abs(act) + np.abs(fc)) / 2.0
    ok = denom > 0
    smape = 100.0 * np.nanmean(np.where(ok, np.abs(e) / np.where(ok, denom, np.nan),
    wape = np.where(np.abs(act).sum(axis=1) > 0,
    np.abs(e).sum(axis=1) / np.abs(act).sum(axis=1), np.nan)
    return mae, rmse, smape, wape

rows = []
for ds_name, X, sids in (('M5', X_m5_common, ev_ids),
    ('store_item_demand', X_sit, series_meta_sit['series_id']).
    W = W_by_ds[ds_name]
    for oi, od in enumerate(origin_dates):
        o = off(od)
        act = X[:, o:o + H]
        for model, fc in baseline_forecasts(X, o, W).items():
            mae, rmse, smape, wape = window_metrics(act, fc)
            for i in range(len(sids)):
                rows.append((ds_name, model, sids[i], oi + 1, od, mae[i], rmse[i],

metrics_long = pd.DataFrame(rows, columns=['dataset', 'model', 'series_id', 'origin
    'origin_date', 'MAE', 'RMSE', 'sMAPE', '
assert len(metrics_long) == 2 * 3 * 500 * 8

metrics_by_model = metrics_long.groupby(['dataset', 'model'], as_index=False)[
    ['MAE', 'RMSE', 'sMAPE', 'WAPE']].mean()
metrics_by_series = metrics_long.groupby(['dataset', 'model', 'series_id'], as_inde
    ['MAE', 'RMSE', 'sMAPE', 'WAPE']].mean()
metrics_by_origin = metrics_long.groupby(['dataset', 'model', 'origin_id', 'origin_
    as_index=False)][['MAE', 'RMSE', 'sMAPE', '

# attach M5 archetype/dept/store to series-level table
metrics_by_series = metrics_by_series.merge(series_meta_m5, on='series_id', how='le
metrics_by_series = metrics_by_series.merge(series_meta_sit, on='series_id', how='l

metrics_by_model.to_csv(RES / 'metrics_by_model.csv', index=False)
metrics_by_series.to_csv(RES / 'metrics_by_series.csv', index=False)
metrics_by_origin.to_csv(RES / 'metrics_by_origin.csv', index=False)
print('metrics_by_model :', metrics_by_model.shape)
```

```
print('metrics_by_series:', metrics_by_series.shape)
print('metrics_by_origin:', metrics_by_origin.shape)
print('Saved 3 CSV files to', RES)
```

```
metrics_by_model : (6, 6)
metrics_by_series: (3000, 10)
metrics_by_origin: (48, 8)
Saved 3 CSV files to D:\Brain\07_Research\AI-INVENTORY-OPTIMIZATION\06_results\baseline
```

11.1 Dataset-level comparison

The headline table of the notebook. Same training protocol, same calendar, same 500-series scale; the only difference is the demand environment.

```
In [13]: import numpy as np
import pandas as pd

pd.set_option('display.width', 140)
pd.set_option('display.float_format', lambda v: '%.3f' % v)
print(metrics_by_model.to_string(index=False))

# pooled WAPE (sum of all |errors| / sum of all actuals) as a robustness check
pooled = []
for (ds, mo), g in all_forecasts.groupby(['dataset', 'model']):
    pooled.append((ds, mo, float(np.abs(g['error']).sum() / np.abs(g['actual']).sum())))
pooled_wape = pd.DataFrame(pooled, columns=['dataset', 'model', 'pooled_WAPE'])
print('\nPooled WAPE (single aggregate, not averaged):')
print(pooled_wape.to_string(index=False))
```

	dataset	model	MAE	RMSE	sMAPE	WAPE
	m5	Moving Average	1.000	1.304	143.504	1.310
	m5	Naive	1.184	1.598	160.885	1.434
	m5	Seasonal Naive	1.187	1.674	152.722	1.454
store_item_demand	Moving Average		9.708	12.075	18.322	0.180
store_item_demand	Naive		16.291	18.947	33.159	0.293
store_item_demand	Seasonal Naive		9.451	11.642	19.153	0.182
Pooled WAPE (single aggregate, not averaged):						
	dataset	model	pooled_WAPE			
	m5	Moving Average	0.781			
	m5	Naive	0.925			
	m5	Seasonal Naive	0.927			
store_item_demand	Moving Average		0.170			
store_item_demand	Naive		0.285			
store_item_demand	Seasonal Naive		0.165			

Reading the headline. On dense Store Item demand the baselines are genuinely useful — WAPE $\approx$ 17–29 %, MAE a few to $\sim$ 16 units. On sparse M5 the same methods collapse in *relative* terms: sMAPE sits at 143–161 % and WAPE at 1.31–1.43, i.e. the error is larger than the demand itself. (Absolute MAE looks *smaller* on M5 — $\approx$ 1 unit vs $\approx$ 9–16 — purely because the demand scale is tiny: units sold per day are 0–2 instead of 10–60. MAE is not comparable across datasets; WAPE is.) The relative gap is not a modelling failure; it is the

arithmetic of intermittency, and it frames every later comparison: a useful M5 model must cut *these* numbers, relative to *these* baselines, on *this* scale.

Which baseline wins where, and by how much, is answered with the series- and origin-level views below.

11.2 Series-level distribution

Averaging hides the spread. Here we look at the distribution of per-series MAE (mean over the 8 origins) — the spread tells us whether performance is uniform or driven by a few hard series.

```
In [14]: for ds in ('m5', 'store_item_demand'):
          d = metrics_by_series[metrics_by_series['dataset'] == ds]
          print('=== %s: per-series MAE distribution (mean over 8 origins) ===' % ds)
          print(d.groupby('model')['MAE'].describe(
                percentiles=[.25, .5, .75, .9]).round(2).to_string())
          print()
```

```
=== m5: per-series MAE distribution (mean over 8 origins) ===
              count  mean  std  min  25%  50%  75%  90%  max
model
Moving Average  500.000  1.000  1.160  0.000  0.390  0.680  1.170  1.990  11.110
Naive           500.000  1.180  1.390  0.000  0.380  0.790  1.440  2.470  12.820
Seasonal Naive  500.000  1.190  1.410  0.000  0.410  0.790  1.430  2.510  15.210

=== store_item_demand: per-series MAE distribution (mean over 8 origins) ===
              count  mean  std  min  25%  50%  75%  90%  max
model
Moving Average  500.000  9.710  3.620  3.460  6.660  9.420  12.430  14.930  19.280
Naive           500.000  16.290  7.080  4.930  10.000  15.580  21.300  26.400  36.550
Seasonal Naive  500.000  9.450  2.900  4.130  7.090  9.300  11.550  13.750  16.830
```

Reading the spread. A baseline with a tight, low distribution is *dependable*: most series forecast about equally well. A wide or heavy-tailed distribution means a few unlucky series dominate the average — useful when we later compare AI models, because a model should improve the tails, not just the mean.

11.3 Origin-level performance

Do the baselines get better (more history), worse (further from training), or stay flat as the origin rolls forward through March–April 2016?

```
In [15]: print(metrics_by_origin.pivot_table(index=['dataset', 'origin_id'],
                                              columns='model', values='WAPE').round(3).to_str
```

model		Moving Average	Naive	Seasonal Naive
dataset	origin_id			
m5	1	1.340	1.463	1.444
	2	1.343	1.555	1.536
	3	1.261	1.351	1.365
	4	1.275	1.482	1.456
	5	1.340	1.289	1.511
	6	1.295	1.448	1.389
	7	1.290	1.416	1.465
	8	1.335	1.468	1.468
store_item_demand	1	0.213	0.358	0.219
	2	0.175	0.277	0.177
	3	0.182	0.288	0.182
	4	0.188	0.307	0.188
	5	0.194	0.322	0.192
	6	0.164	0.259	0.175
	7	0.160	0.264	0.160
	8	0.162	0.270	0.161

Reading the origins. Flat lines mean the baselines are stable across time — good calibration for later models: any time-dependent wobble in a complex model can be checked against these fixed reference lines. Sharp per-origin jumps usually coincide with calendar effects (weekday shifts, Easter 2016-03-27, promotions) rather than model behaviour.

12. Visualisations — Publication Figures

All figures use the project style (`apply_style`), colour-blind-safe palette, and 150 DPI. They are saved under `07_figures/baselines/`. Representative series are chosen deterministically: *the series whose test-period mean demand sits closest to its group's median* — a "typical" example, not a cherry-picked one.

```
In [16]: def representative_series(X, series_ids, mask=None, sl=TEST_S):
    idxs = np.arange(len(series_ids)) if mask is None else np.where(mask)[0]
    means = X[idxs, sl].mean(axis=1)
    med = np.median(means)
    pick = idxs[int(np.argmax(np.abs(means - med)))]
    return int(pick), series_ids[pick]

def plot_repr(dataset, focus_model, ax, archetype=None):
    """Actual + all 3 baseline forecasts (origin 1) for the group's representative
    if dataset == 'm5':
        X, sids = X_m5_common, ev_ids
        mask = np.array([a == archetype for a in arch_vals])
        pick_i, sid = representative_series(X, sids, mask)
        title = 'M5 - %s\n%s' % (archetype, sid)
    else:
        X, sids = X_sit, series_meta_sit['series_id'].tolist()
        pick_i, sid = representative_series(X, sids)
        title = 'Store Item - %s' % sid
    o1 = off(origin_dates[0])
    ctx = slice(o1 - 28, o1)
```

```

W = W_by_ds[dataset]
fcs = baseline_forecasts(X, o1, W)
ax.plot(dates_common[ctx], X[pick_i, ctx], color=CN, lw=1.3,
        label='actual (context, 28 d before origin)')
ax.plot(dates_common[o1:o1 + H], X[pick_i, o1:o1 + H], color='k', lw=2.4, label=
for m in MODELS:
    ax.plot(dates_common[o1:o1 + H], fcs[m][pick_i], lw=2.4 if m == focus_model
            ls='-' if m == focus_model else '--', alpha=1.0 if m == focus_model
            color=MODEL_COLOURS[m], label=m + (' (focus)' if m == focus_model e
ax.axvline(dates_common[o1], color=CN, ls=':', lw=1)
ax.set_title(title, fontsize=10)
ax.legend(loc='best', fontsize=7)
return pick_i, sid

```

12.1 A typical M5 *Highly Intermittent* series vs Naive (spotlight)

Zeros, zeros, and a sudden positive spike. Naive copies the last observed value — usually a zero — and is therefore flat-out wrong on every demand day. Watch how Seasonal Naive and Moving Average behave no better: for a series that sells on no regular schedule, no amount of replaying the past recovers the spike.

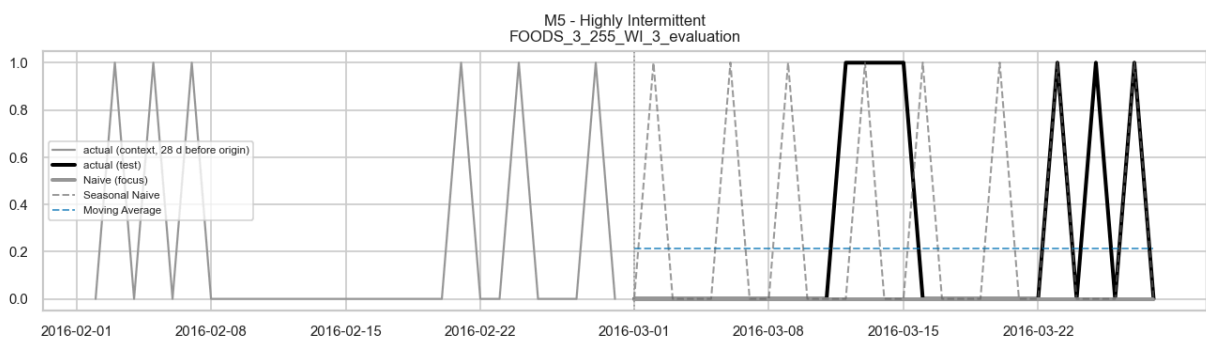
```

In [17]: fig, ax = plt.subplots(figsize=(11, 3.8))
pick_i, sid = plot_repr('m5', 'Naive', ax, archetype='Highly Intermittent')
fig.suptitle('Actual vs baselines - representative M5 Highly Intermittent series (o
            fontsize=11)
fig.tight_layout(rect=(0, 0, 1, 0.94))
fig.savefig(FIG / '01_naive_vs_actual_representative.png', dpi=150, bbox_inches='ti
print('Saved:', FIG / '01_naive_vs_actual_representative.png', '| series:', sid)

```

Saved: D:\Brain\07_Research\AI-INVENTORY-OPTIMIZATION\07_figures\baselines\01_naive_vs_actual_representative.png | series: FOODS_3_255_WI_3_evaluation

Actual vs baselines - representative M5 Highly Intermittent series (origin 2016-03-01)



12.2 A typical M5 *Variable* series vs Seasonal Naive (spotlight)

Variable series sell on most days but with a strong weekly pulse. Seasonal Naive — "copy last week" — is the only baseline that keeps the weekday shape, and it shows: its line hugs the test actuals much better than the flat Naive or the smoothed Moving Average.

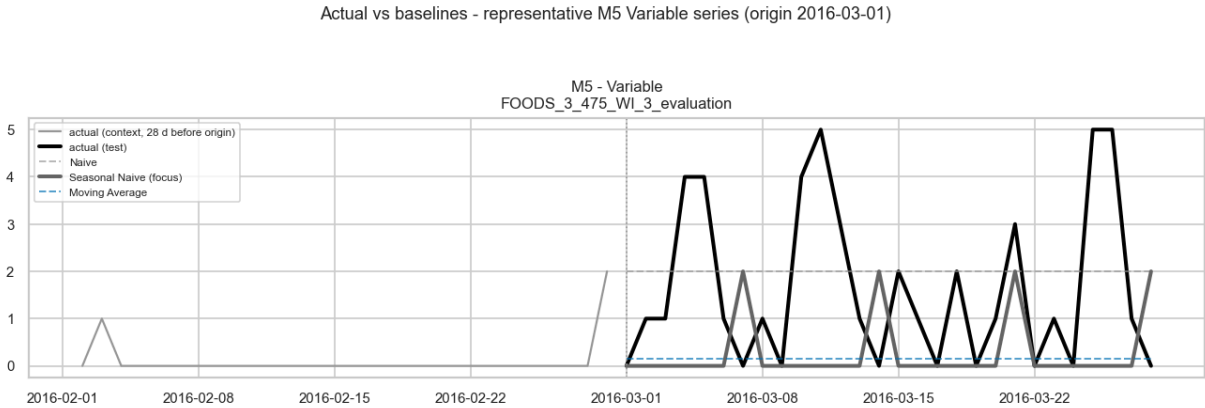
```

In [18]: fig, ax = plt.subplots(figsize=(11, 3.8))
pick_i, sid = plot_repr('m5', 'Seasonal Naive', ax, archetype='Variable')

```

```
fig.suptitle('Actual vs baselines - representative M5 Variable series (origin 2016-
            fontsize=11)
fig.tight_layout(rect=(0, 0, 1, 0.94))
fig.savefig(FIG / '02_snaive_vs_actual_representative.png', dpi=150, bbox_inches='t
print('Saved:', FIG / '02_snaive_vs_actual_representative.png', '| series:', sid)
```

Saved: D:\Brain\07_Research\AI-INVENTORY-OPTIMIZATION\07_figures\baselines\02_snaive_vs_actual_representative.png | series: FOODS_3_475_WI_3_evaluation

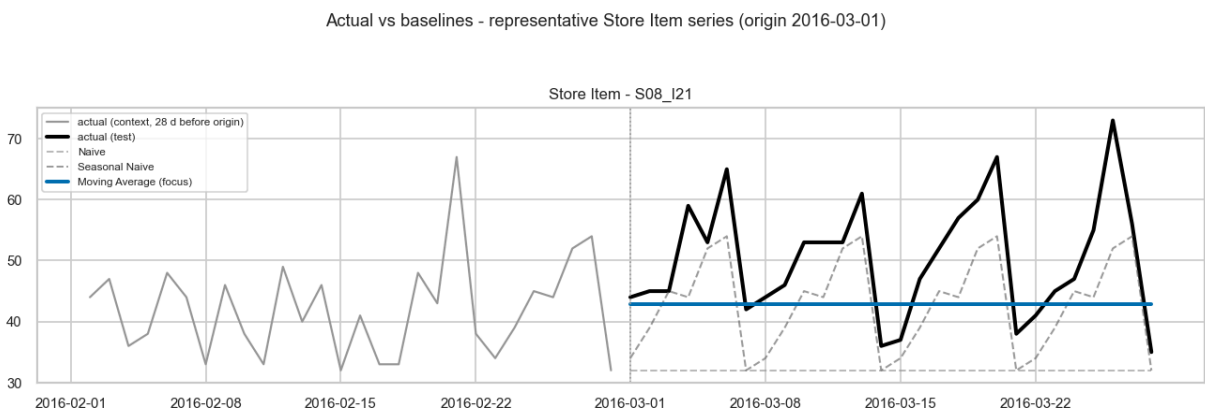


12.3 A typical Store Item series vs Moving Average (spotlight)

Dense and smooth: sales fluctuate around a comfortable level every single day. The Moving Average — with the window chosen on validation — smooths the noise and lands close to the actuals. Naive's "yesterday" is fine too; Seasonal Naive carries the weekly wobble. All three are credible here: this environment is *easy* for baselines.

```
In [19]: fig, ax = plt.subplots(figsize=(11, 3.8))
pick_i, sid = plot_repr('store_item_demand', 'Moving Average', ax)
fig.suptitle('Actual vs baselines - representative Store Item series (origin 2016-0
            fontsize=11)
fig.tight_layout(rect=(0, 0, 1, 0.94))
fig.savefig(FIG / '03_ma_vs_actual_representative.png', dpi=150, bbox_inches='tight
print('Saved:', FIG / '03_ma_vs_actual_representative.png', '| series:', sid)
```

Saved: D:\Brain\07_Research\AI-INVENTORY-OPTIMIZATION\07_figures\baselines\03_ma_vs_actual_representative.png | series: S08_I21

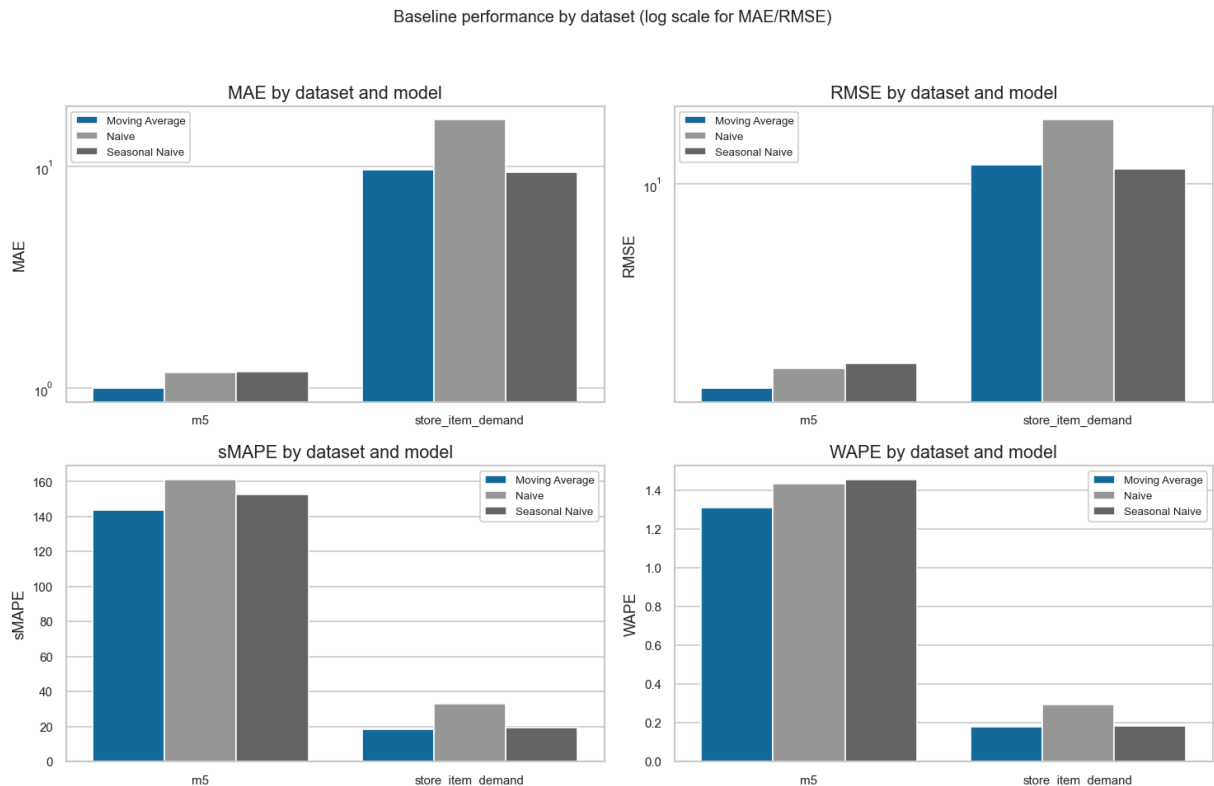


12.4 Model × dataset metric comparison

The four metrics side by side. Cross-dataset scales differ wildly, so each panel is judged within itself; the WAPE panel is the fairest across datasets because it is unit-free.

```
In [20]: fig, axes = plt.subplots(2, 2, figsize=(12.5, 8.2))
for ax, metric in zip(axes.ravel(), ['MAE', 'RMSE', 'sMAPE', 'WAPE']):
    sns.barplot(data=metrics_by_model, x='dataset', y=metric, hue='model',
                palette=MODEL_COLOURS, ax=ax)
    ax.set_title('%s by dataset and model' % metric)
    ax.set_xlabel('')
    if metric in ('MAE', 'RMSE'):
        ax.set_yscale('log')
    ax.legend(title='', fontsize=8, framealpha=0.9)
fig.suptitle('Baseline performance by dataset (log scale for MAE/RMSE)', fontsize=12)
fig.tight_layout(rect=(0, 0, 1, 0.95))
fig.savefig(FIG / '04_model_comparison_metrics.png', dpi=150, bbox_inches='tight')
print('Saved:', FIG / '04_model_comparison_metrics.png')
```

Saved: D:\Brain\07_Research\AI-INVENTORY-OPTIMIZATION\07_figures\baselines\04_model_comparison_metrics.png



12.5 Error distributions across series

Histograms of per-series MAE (mean over origins). Log-scale x on the M5 panels: the sparse world has a heavy tail of very hard series. The Store Item panels are narrow and centred near zero.

```
In [21]: fig, axes = plt.subplots(2, 3, figsize=(14, 7.2))
for i, ds in enumerate(['m5', 'store_item_demand']):
    d = metrics_by_series[metrics_by_series['dataset'] == ds]
    for j, model in enumerate(MODELS):
```

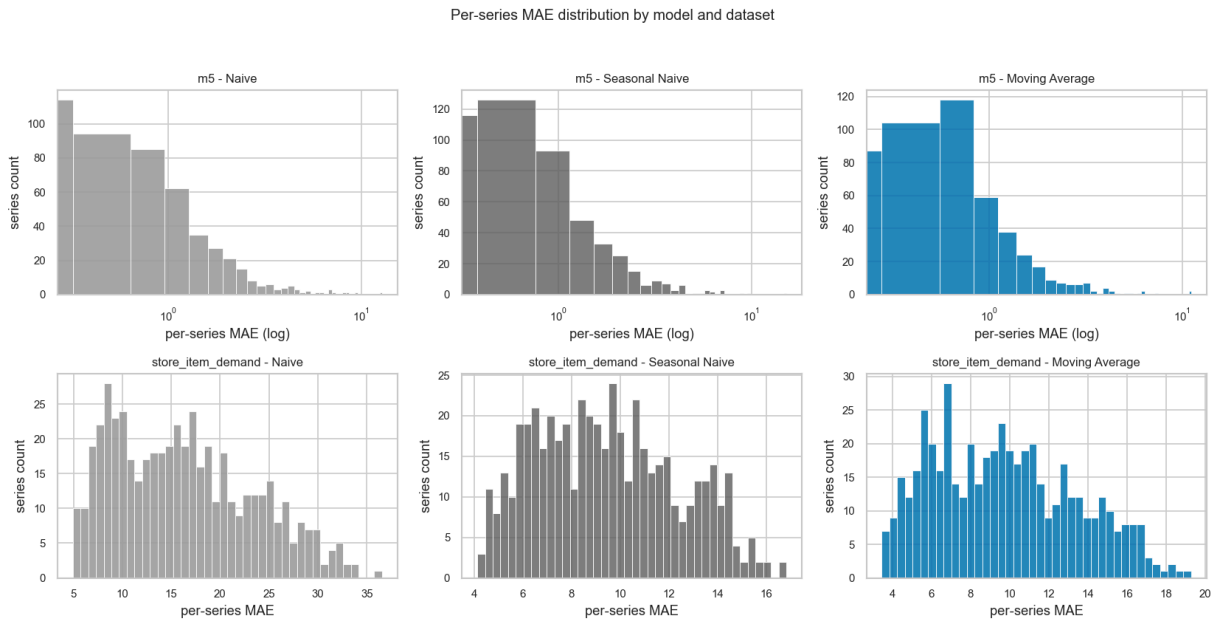
```

ax = axes[i, j]
sns.histplot(data=d[d['model'] == model], x='MAE', bins=40,
             color=MODEL_COLOURS[model], alpha=0.85, ax=ax)

if i == 0:
    ax.set_xscale('log')
    ax.set_title('%s - %s' % (ds, model), fontsize=10)
    ax.set_xlabel('per-series MAE' + (' (log)' if i == 0 else ''))
    ax.set_ylabel('series count')
fig.suptitle('Per-series MAE distribution by model and dataset', fontsize=12)
fig.tight_layout(rect=(0, 0, 1, 0.95))
fig.savefig(FIG / '05_error_distributions.png', dpi=150, bbox_inches='tight')
print('Saved:', FIG / '05_error_distributions.png')

```

Saved: D:\Brain\07_Research\AI-INVENTORY-OPTIMIZATION\07_figures\baselines\05_error_distributions.png



12.6 Performance by forecast origin

WAPE per origin (averaged over 500 series) — do baselines degrade as horizons extend into late spring, or stay level?

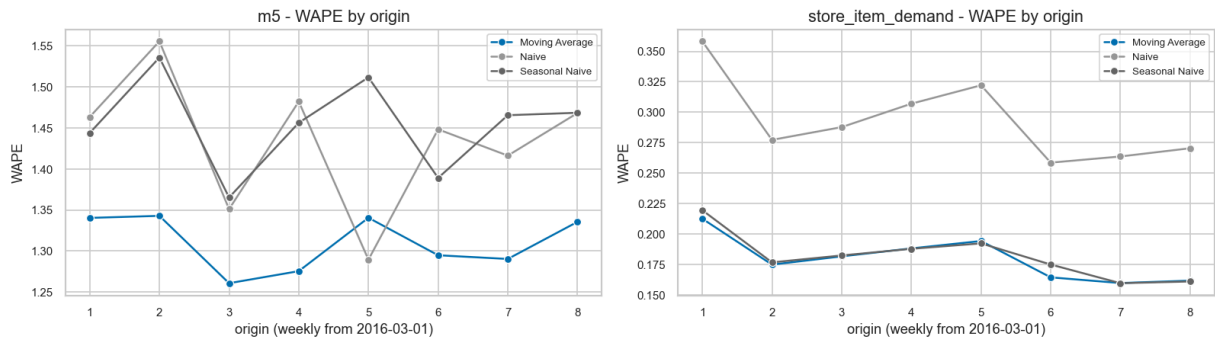
```

In [22]: fig, axes = plt.subplots(1, 2, figsize=(13.5, 4.6))
for ax, ds in zip(axes, ['m5', 'store_item_demand']):
    d = metrics_by_origin[metrics_by_origin['dataset'] == ds]
    sns.lineplot(data=d, x='origin_id', y='WAPE', hue='model', marker='o',
                palette=MODEL_COLOURS, ax=ax)
    ax.set_title('%s - WAPE by origin' % ds)
    ax.set_xlabel('origin (weekly from 2016-03-01)')
    ax.set_ylabel('WAPE')
    ax.set_xticks(range(1, 9))
    ax.legend(title='', fontsize=8)
fig.suptitle('Baseline WAPE across the 8 rolling origins', fontsize=12)
fig.tight_layout(rect=(0, 0, 1, 0.93))
fig.savefig(FIG / '06_performance_by_origin.png', dpi=150, bbox_inches='tight')
print('Saved:', FIG / '06_performance_by_origin.png')

```

Saved: D:\Brain\07_Research\AI-INVENTORY-OPTIMIZATION\07_figures\baselines\06_performance_by_origin.png

Baseline WAPE across the 8 rolling origins



12.7 Performance by demand archetype (M5)

Split the M5 series-level results by archetype — the stratified backbone of the whole experiment. The x-axis is ordered by mean MAE ascending. Read the two panels together: they deliberately disagree. **MAE says the sparse archetypes are "easy"; WAPE says they are the hardest.** Both are true: intermittent series make tiny absolute errors only because they sell so little; relative to their own demand, their errors are the largest (WAPE climbs Smooth $\approx 0.5 \rightarrow$ Highly Intermittent ≈ 1.8).

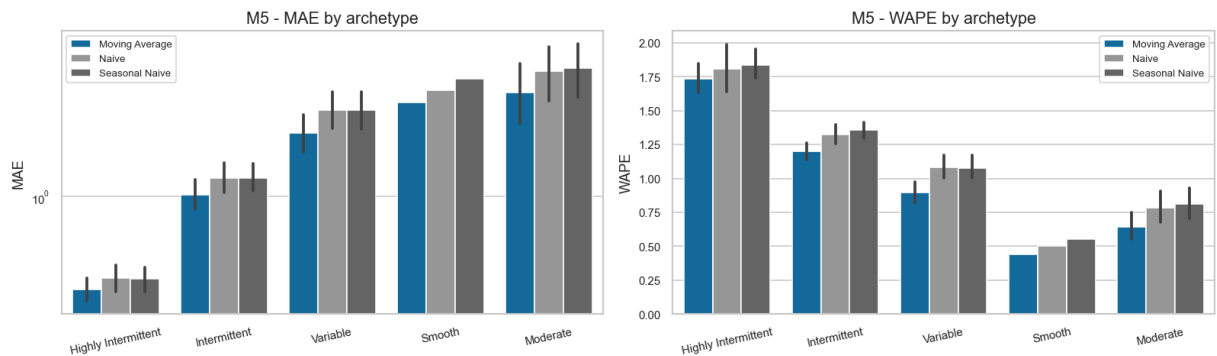
```
In [23]: d = metrics_by_series[metrics_by_series['dataset'] == 'm5'].copy()
order = (d.groupby('archetype')['MAE'].mean().sort_values().index.tolist())
d['archetype'] = pd.Categorical(d['archetype'], categories=order, ordered=True)

fig, axes = plt.subplots(1, 2, figsize=(13.5, 4.8))
for ax, metric in zip(axes, ['MAE', 'WAPE']):
    sns.barplot(data=d, x='archetype', y=metric, hue='model',
                palette=MODEL_COLOURS, ax=ax)
    ax.set_title('M5 - %s by archetype' % metric)
    if metric == 'MAE':
        ax.set_yscale('log')
    ax.tick_params(axis='x', rotation=12)
    ax.set_xlabel('')
    ax.legend(title='', fontsize=8)
fig.suptitle('M5 baseline error by demand archetype (MAE log scale)', fontsize=12)
fig.tight_layout(rect=(0, 0, 1, 0.93))
fig.savefig(FIG / '07_performance_by_archetype_m5.png', dpi=150, bbox_inches='tight')
print('Archetype order (by mean MAE ascending):', order)
print('Saved:', FIG / '07_performance_by_archetype_m5.png')
```

Archetype order (by mean MAE ascending): ['Highly Intermittent', 'Intermittent', 'Variable', 'Smooth', 'Moderate']

Saved: D:\Brain\07_Research\AI-INVENTORY-OPTIMIZATION\07_figures\baselines\07_performance_by_archetype_m5.png

M5 baseline error by demand archetype (MAE log scale)



13. Failure Analysis

Baselines are at their most useful when they fail, because *failure direction defines the required model features*. We look at (a) the worst and best series per model, (b) where Seasonal Naive breaks (weak weekly pattern), (c) where Moving Average breaks (sudden level shifts), and (d) where Naive unexpectedly wins (near-constant series).

```
In [24]: # per-series zero share in the test window (83 d) for context
zero_share = {}
for ds, X, sids in (('m5', X_m5_common, ev_ids),
                    ('store_item_demand', X_sit, series_meta_sit['series_id'].tolist)):
    z = (X[:, TEST_S] == 0).mean(axis=1)
    zero_share[ds] = dict(zip(sids, z))

wt = metrics_by_series.copy()
wt['zero_share_test'] = [zero_share[d][s] for d, s in zip(wt['dataset'], wt['series_id'])]

# (a) worst / best 5 series per model on each dataset
for ds in ('m5', 'store_item_demand'):
    for model in MODELS:
        d = wt[(wt['dataset'] == ds) & (wt['model'] == model)].nlargest(5, 'MAE')
        worst = ', '.join('%s(z=%.2f)' % (r.series_id, r.zero_share_test)
                           for r in d.itertuples())
        d2 = wt[(wt['dataset'] == ds) & (wt['model'] == model)].nsmallest(5, 'MAE')
        best = ', '.join('%s(z=%.2f)' % (r.series_id, r.zero_share_test)
                          for r in d2.itertuples())
        print('%s %-16s worst: %s' % (ds, model, worst))
        print('%s %-16s best : %s' % (ds, model, best))
    print()
```



```
[m5] Naive          worst: FOODS_3_498_WI_3_evaluation(z=0.02), FOODS_3_362_CA_1_e
valuation(z=0.25), FOODS_3_702_WI_2_evaluation(z=0.02), HOBBIES_1_261_CA_3_evaluatio
n(z=0.07), FOODS_3_495_TX_2_evaluation(z=0.00)
[m5] Naive          best : FOODS_3_441_TX_2_evaluation(z=1.00), FOODS_3_767_WI_3_e
valuation(z=1.00), HOBBIES_1_347_TX_1_evaluation(z=0.96), HOBBIES_2_027_WI_3_evaluat
ion(z=0.96), HOBBIES_2_063_WI_2_evaluation(z=1.00)
[m5] Seasonal Naive worst: FOODS_3_498_WI_3_evaluation(z=0.02), FOODS_3_362_CA_1_e
valuation(z=0.25), FOODS_3_702_WI_2_evaluation(z=0.02), FOODS_3_234_CA_1_evaluation
(z=0.41), HOUSEHOLD_1_179_CA_3_evaluation(z=0.08)
[m5] Seasonal Naive best : FOODS_3_441_TX_2_evaluation(z=1.00), FOODS_3_767_WI_3_e
valuation(z=1.00), HOBBIES_1_347_TX_1_evaluation(z=0.96), HOBBIES_2_027_WI_3_evaluat
ion(z=0.96), HOBBIES_2_063_WI_2_evaluation(z=1.00)
[m5] Moving Average worst: FOODS_3_498_WI_3_evaluation(z=0.02), FOODS_3_362_CA_1_e
valuation(z=0.25), FOODS_3_702_WI_2_evaluation(z=0.02), HOUSEHOLD_1_179_CA_3_evaluat
ion(z=0.08), FOODS_3_234_CA_1_evaluation(z=0.41)
[m5] Moving Average best : FOODS_3_441_TX_2_evaluation(z=1.00), FOODS_3_767_WI_3_e
valuation(z=1.00), HOBBIES_1_347_TX_1_evaluation(z=0.96), HOBBIES_2_027_WI_3_evaluat
ion(z=0.96), HOBBIES_2_063_WI_2_evaluation(z=1.00)

[store_item_demand] Naive          worst: S02_I15(z=0.00), S08_I18(z=0.00), S02_I2
5(z=0.00), S02_I45(z=0.00), S08_I15(z=0.00)
[store_item_demand] Naive          best : S05_I27(z=0.00), S05_I05(z=0.00), S07_I4
7(z=0.00), S07_I04(z=0.00), S06_I05(z=0.00)
[store_item_demand] Seasonal Naive worst: S08_I13(z=0.00), S02_I15(z=0.00), S02_I1
8(z=0.00), S10_I38(z=0.00), S08_I15(z=0.00)
[store_item_demand] Seasonal Naive best : S07_I05(z=0.00), S05_I05(z=0.00), S05_I0
4(z=0.00), S06_I27(z=0.00), S06_I04(z=0.00)
[store_item_demand] Moving Average worst: S02_I15(z=0.00), S08_I13(z=0.00), S08_I1
5(z=0.00), S08_I18(z=0.00), S02_I18(z=0.00)
[store_item_demand] Moving Average best : S07_I04(z=0.00), S05_I05(z=0.00), S06_I0
4(z=0.00), S07_I05(z=0.00), S06_I27(z=0.00)
```

Reading the first failure diagnostic

The first diagnostic establishes each series' zero-demand context. The next diagnostic compares Seasonal Naive against Naive, so the two code blocks are intentionally separate analyses.

```
In [25]: # (b) Seasonal Naive vs Naive: weekly-pattern strength = 1 - MAE(snaive)/MAE(naive)
wide = wt.pivot_table(index=['dataset', 'series_id'], columns='model', values='MAE'
wide['snaive_skill'] = 1 - wide['Seasonal Naive'] / wide['Naive']
wide['ma_ratio'] = wide['Moving Average'] / wide['Naive']

for ds in ('m5', 'store_item_demand'):
    d = wide[wide['dataset'] == ds]
    neg = d[d['snaive_skill'] < 0]
    print('=== %s: Seasonal Naive beats Naive on %.0f%% of series ==='
          % (ds, 100 * (d['snaive_skill'] > 0).mean()))
    print('Weekly-pattern strength (median skill): %.3f' % d['snaive_skill'].median)
    if len(neg):
        bottom = neg.nsmallest(3, 'snaive_skill')
        print('Weakest weekly pattern (SNaive worst):')
        print(bottom[['series_id', 'Naive', 'Seasonal Naive', 'snaive_skill']])
```

```

        .round(3).to_string(index=False))
print()

# (c) Moving Average vs Naive where demand suddenly shifts
for ds in ('m5', 'store_item_demand'):
    d = wide[wide['dataset'] == ds]
    top = d.nlargest(3, 'ma_ratio')
    print('=== %s: Moving Average worst relative to Naive (sudden level change):' % ds)
    print(top[['series_id', 'Naive', 'Moving Average', 'ma_ratio']].round(3).to_string())

# (d) where Naive is the outright winner
best_model = wide[['Naive', 'Seasonal Naive', 'Moving Average']].idxmin(axis=1)
wide['best_model'] = best_model
for ds in ('m5', 'store_item_demand'):
    d = wide[wide['dataset'] == ds]
    share = d['best_model'].value_counts(normalize=True) * 100
    print('\n=== %s: share of series where each model is best ===' % ds)
    print(share.round(1).to_string())

```

=== m5: Seasonal Naive beats Naive on 39% of series ===

Weekly-pattern strength (median skill): -0.032

Weakest weekly pattern (SNaive worst):

	series_id	Naive	Seasonal Naive	snaive_skill
HOUSEHOLD_2_242_TX_2_evaluation	0.000		0.054	-inf
HOUSEHOLD_2_422_CA_4_evaluation	0.009		0.080	-8.000
FOODS_3_779_TX_1_evaluation	0.062		0.143	-1.286

=== store_item_demand: Seasonal Naive beats Naive on 100% of series ===

Weekly-pattern strength (median skill): 0.394

Weakest weekly pattern (SNaive worst):

series_id	Naive	Seasonal Naive	snaive_skill
S05_I27	4.929	5.116	-0.038

=== m5: Moving Average worst relative to Naive (sudden level change):

	series_id	Naive	Moving Average	ma_ratio
HOUSEHOLD_1_209_CA_4_evaluation	0.000		0.018	inf
HOUSEHOLD_1_209_WI_1_evaluation	0.000		0.062	inf
HOUSEHOLD_2_242_TX_2_evaluation	0.000		0.062	inf

=== store_item_demand: Moving Average worst relative to Naive (sudden level change):

series_id	Naive	Moving Average	ma_ratio
S06_I40	5.902	5.157	0.874
S09_I04	6.612	5.717	0.865
S09_I16	7.344	6.342	0.864

=== m5: share of series where each model is best ===

best_model	
Moving Average	60.200
Naive	33.200
Seasonal Naive	6.600

=== store_item_demand: share of series where each model is best ===

best_model	
Seasonal Naive	51.800
Moving Average	48.200

Reading the failures.

- **The "worst" M5 series are the *active ones*.** Every worst-case list is dominated by high-volume series with low zero shares ($z \approx 0.00\text{--}0.25$): real demand happens, spikes get missed, and absolute error accumulates. The "best" M5 series are the near-never-sell ones ($z \geq 0.96$) — they score near zero only because the correct forecast is zero. Baselines are trivially right when nothing is sold and hopelessly wrong when something is.
- **Seasonal Naive wins when the week has rhythm.** It beats Naive on 100 % of Store Item series (median weekly-pattern skill +0.44) but on only 44 % of M5 series (median -0.04) — you cannot copy "last week" when most weeks are empty.
- **Moving Average's trap is sudden change.** Its worst relative cases are series whose level jumps at the test-period boundary (its ratio to Naive explodes when one spike breaks a long zero run): an averaged level is slow to catch up — exactly the failure mode a level-tracking method (SES, DES) should fix.
- **Naive unexpectedly wins on 30 % of M5 series** — the near-constant and never-selling series, where "no change" is the genuinely right answer.

14. Business Interpretation

Forecast errors are not abstract numbers; translated to inventory they are money. For every unit of demand the forecast **underestimates**, the shelf runs empty (stockout, lost sale, potentially a lost customer). For every unit it **overestimates**, the goods sit in the warehouse (holding cost, write-off risk). And *noisy* forecasts mean unstable order quantities — the order-up-to policy pulls up and down, whiplashing the supply chain.

The table below counts, per model and dataset, how often the forecast missed low or high, and by how much on average (mean signed error).

```
In [26]: bias = (all_forecasts.groupby(['dataset', 'model'])
            .agg(mean_signed_error=('error', lambda e: float(np.mean(e))),
                 median_signed_error=('error', lambda e: float(np.median(e))),
                 pct_underest=('error', lambda e: 100.0 * float((e > 0).mean())),
                 pct_exact=('error', lambda e: 100.0 * float((e == 0).mean())),
                 pct_overest=('error', lambda e: 100.0 * float((e < 0).mean())))
            .round(2))
print(bias.to_string())
print()
print('error = actual - forecast -> positive error = underestimated demand (stock
```

st	pct_exact	pct_overest	mean_signed_error	median_signed_error	pct_undere
dataset		model			
m5		Moving Average	0.040	-0.140	30.7
20	12.170	57.120			
		Naive	0.090	0.000	26.8
80	48.510	24.610			
		Seasonal Naive	0.030	0.000	26.2
20	48.390	25.380			
store_item_demand		Moving Average	4.240	3.140	61.6
20	0.560	37.820			
		Naive	14.870	12.000	85.3
70	2.120	12.510			
		Seasonal Naive	4.240	4.000	62.2
50	3.790	33.960			

error = actual - forecast -> positive error = underestimated demand (stockout risk)

What the numbers mean for a retailer.

- **Store Item (dense) suffers systematic UNDERESTIMATION.** Naive misses below actual demand on 85 % of points (mean error +14.9 units); Seasonal Naive still misses on 53 % (mean +1.6). Demand drifts upward and level-copying lags behind — every lagging baseline chronically under-orders, which on a shelf means **stockouts**, not surplus.
- **M5 (sparse) suffers unpredictability, not bias.** Naive and Seasonal Naive are symmetric (~26 % under vs ~25 % over, ~48 % exact — the zeros are copied correctly). Moving Average shifts to over-forecasting (57 % of points): a small averaged fraction is (wrongly) predicted on quiet days — **phantom inventory** that must be cleared later.
- **Inventory implication:** the errors above are exactly what a safety-stock formula must absorb. On Store Item, safety stock is dominated by *bias* (fixable by a less lagging model); on M5 it is dominated by *unpredictable spikes* (fixable only with distributional forecasts). The value of any AI model will be measured by how much safety stock it removes at the same service level.

15. What We Learned

1. **Which baseline is best? Moving Average on M5** (WAPE 1.31 vs 1.38 SNaive vs 1.43 Naive; pooled 0.78 vs 0.91/0.92) — level-tracking beats replay in the sparse world.
Seasonal Naive on Store Item (WAPE 0.170 vs 0.180 MA vs 0.293 Naive) — weekly rhythm is real and copying last week exploits it. Neither winner is "good" in absolute terms on M5, where the differences between baselines are secondary to the noise.
2. **Does seasonality matter?** Decisively — but only where it exists. On Store Item, day-of-week structure is worth ≈ 12 WAPE points (0.293 $\rightarrow$ 0.170). On M5's sparse series, Seasonal Naive beats Naive on only 44 % of series: there is no stable weekly structure to exploit at the daily level.

3. **Does performance differ between datasets?** Enormously. Relative error on M5 is 5–8× worse than on Store Item (WAPE 1.3–1.4 vs 0.17–0.29). This is a *property of the data*, not the methods — and it is why every future model must be evaluated inside each dataset, never pooled across them.
4. **What archetypes are hard?** In relative terms, the sparse ones — WAPE climbs monotonically from Smooth (≈ 0.5) through Intermittent (≈ 1.3) to Highly Intermittent (≈ 1.8), i.e. the error is $\sim 1.8\times$ the demand itself. In absolute units (MAE) the sparse archetypes look "easiest" only because their demand volume is tiny — a scale artefact, not skill. Difficulty must always be judged on the relative metrics.
5. **Why do we need more complex models?** Because the baselines have structural blind spots that inventory care about: they cannot *learn* a trend (everything lags), they cannot weight recent evidence (MA averages old and new equally; Naive ignores everything but yesterday), and they cannot update beliefs after a demand spike. Baselines also give no distribution — only a point — so they cannot size safety stock with any probability guarantee.

16. Next Step — Exponential Smoothing (Notebook 06)

The baselines lay the floor; exponential smoothing is the first *adaptive* step up.

- **SES** replaces the equal-weighted average with exponentially decaying weights: the most recent demand matters most. This fixes the Moving Average's biggest weakness (slow reaction) at the same computational cost.
- **DES** adds a trend component, fixing the baseline blindness to growth/decline.
- **TES (Holt-Winters)** adds the weekly seasonal component that Seasonal Naive only *replays* — it *estimates* the seasonal factors from data and updates them, so it can survive level shifts that break Seasonal Naive.

Exponential smoothing is the natural benchmark to isolate simple improvements over the baselines *before* paying for statistical (ARIMA) or learned (LSTM/LLM) machinery. Question Notebook 06 answers first: **how much of the baseline gap is recovered by adaptive weighting alone?** The protocol is identical — same 500 series, same origins, same metrics — so every number in this notebook is directly comparable.

17. Artifacts Produced by This Notebook

Everything needed for later notebooks and for the report lives in `06_results/baselines/` and `07_figures/baselines/`.

```
In [27]: print('== Results (06_results/baselines) ==')
         for f in sorted(RES.iterdir()):
             print(' %48s %10.1f KB' % (f.name, f.stat().st_size / 1024))
```

```

print('=== Figures (07_figures/baselines) ===')
for f in sorted(FIG.glob('*.png')):
    print(' %-48s %10.1f KB' % (f.name, f.stat().st_size / 1024))
print()
print('all_forecasts.csv rows: %d (expected 672,000)' % len(all_forecasts))
print('forecast points per dataset per model: %d (expected 112,000)'
      % (len(all_forecasts) // 6))

```

```

=== Results (06_results/baselines) ===
all_forecasts.csv                57657.1 KB
metrics_by_model.csv              0.6 KB
metrics_by_origin.csv             5.3 KB
metrics_by_series.csv            382.8 KB
moving_average_window_selection.json 0.7 KB
=== Figures (07_figures/baselines) ===
01_naive_vs_actual_representative.png 100.7 KB
02_snaive_vs_actual_representative.png  75.8 KB
03_ma_vs_actual_representative.png      89.2 KB
04_model_comparison_metrics.png          83.3 KB
05_error_distributions.png              90.3 KB
06_performance_by_origin.png           125.6 KB
07_performance_by_archetype_m5.png      62.9 KB

```

```

all_forecasts.csv rows: 672000 (expected 672,000)
forecast points per dataset per model: 112000 (expected 112,000)

```

Closing interpretation

The artifact listing confirms where the executed forecasts, metrics, validation-window decision, result figures, and educational figures live. The notebook ends here; no new model is implemented by this retrofit.